PHY4605 Computational Methods in Physics
Learning Note | Week 2

# Pseudocode, Loops, and Debugging

*Turn one familiar physics equation into a sequence that can be traced, checked, and repaired.*

**Physical question.** A ball is launched straight upward. We already know its position equation. How can we make MATLAB calculate one position at a time in an order that another person can trace and verify?

## Core learning outcomes

1. decompose a familiar physics calculation into short pseudocode;
2. trace, complete, and modify a simple indexed `for` loop; and
3. diagnose a clear defect using code evidence, one known-value check, and physical reasoning.

## How to use this note

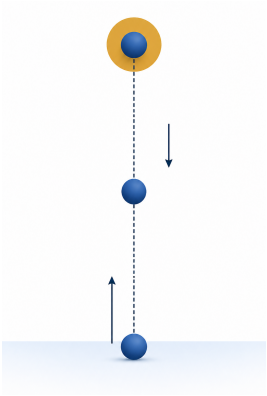
Read the physical claim first, then the pseudocode, then the MATLAB fragment. Pause at each trace or prediction checkpoint before looking at the supplied result. The aim is not to memorise loop punctuation; it is to make every computational step explainable.

## 1. Reuse a familiar physical model

Choose upward as positive, neglect air resistance, and take gravity to be constant. The vertical position is

$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2.$$

In words: *position equals the starting position plus the upward launch contribution minus the downward gravity contribution.*



Familiar vertical launch: the geometry is one-dimensional, so the computational focus can stay on the algorithm.

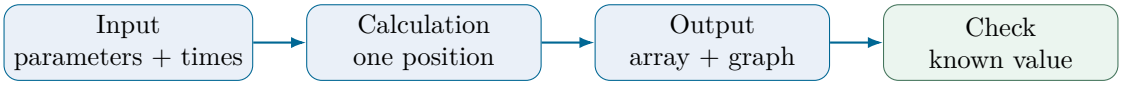
| Physics symbol | MATLAB name | Meaning and unit |
|---|---|---|
| $y(t)$ | `y_m` | vertical position, m |
| $t$ | `t_s` | ordered time samples, s |
| $y_0$ | `y0_m` | starting position, m |
| $v_0$ | `v0_mps` | upward launch velocity, m $s^{-1}$ |
| $g$ | `g_mps2` | gravity magnitude, m $s^{-2}$ |

Use $y_0 = 0\,\mathrm{m}$, $v_0 = 20\,\mathrm{m\,s^{-1}}$, $g = 9.81\,\mathrm{m\,s^{-2}}$, and times from 0 to 4 s in 0.5 s steps.

**Prediction checkpoint.** The ball should rise, slow, reach a highest point, and then fall. At $t = 0$, the model must return $y = 0\,\mathrm{m}$.

## 2. Decompose before writing MATLAB

Separate the task into four jobs before thinking about syntax.




This decomposition is already an algorithm map. It answers *what must happen* before MATLAB answers *how to write it*.

## 3. Write guided pseudocode

One sensible pseudocode sequence is:

1. store $y_0$, $v_0$, $g$, and the ordered time samples;
2. make one empty position slot for every time;
3. start at the first MATLAB index;
4. read the current time;
5. calculate one position and store it in the matching slot;
6. move to the next index and repeat until the final time; and
7. plot, check, and interpret the result.

**Key idea.** Pseudocode is not a rough version of MATLAB punctuation. It is a readable description of the calculation order.

## 4. A loop is a controlled repeated update

The time array and parameters are declared before the loop.

Listing 1: Locked inputs for the lecture example

```
y0_m = 0;
v0_mps = 20;
g_mps2 = 9.81;
t_s = 0:0.5:4;
n_samples = numel(t_s);
```

Preallocation creates the matching output array before the loop starts:

Listing 2: Preallocate one storage slot per time sample

```
y_m = zeros(size(t_s));
```

Inside the loop, the currently selected time is a scalar. Ordinary scalar multiplication and powers are therefore appropriate.

Listing 3: One position is calculated and stored on each pass

```
for sample_index = 1:n_samples
    current_time_s = t_s(sample_index);
    y_m(sample_index) = y0_m + v0_mps*current_time_s ...
        - 0.5*g_mps2*current_time_s^2;
end
```

The five pieces to look for are: **start**, **current value**, **update**, **matching storage location**, and **stopping point**.

## 5. Trace the first two passes by hand

Do not begin by staring at the whole loop. Follow one index at a time.

| Pass | Index | Current time | Stored position |
|---|---|---|---|
| 1 | `sample_index=1` | `t_s(1)=0.0 s` | `y_m(1)=0.00000 m` |
| 2 | `sample_index=2` | `t_s(2)=0.5 s` | `y_m(2)=8.77375 m` |

**Trace checkpoint.** Why does index 2 mean 0.5 s rather than 2 s? Because MATLAB indexing counts storage locations; the stored values are set by the time array.

## 6. Bounded completion instead of blank-page coding

Suppose the loop scaffold is supplied as

Listing 4: Complete only the missing calculation

```
for sample_index = 1:n_samples
    current_time_s = t_s(sample_index);
    y_m(sample_index) = _______________________________;
end
```

Translate the current step back into words: *use the current time, calculate one vertical position, and store it in the matching location.* The completed expression is

```
y0_m + v0_mps*current_time_s - 0.5*g_mps2*current_time_s^2
```

## 7. Read the output, then modify one input

The retained MATLAB evidence uses exactly the same loop with $v_0 = 20\,\mathrm{m\,s^{-1}}$ and then with only the launch speed changed to $15\,\mathrm{m\,s^{-1}}$.

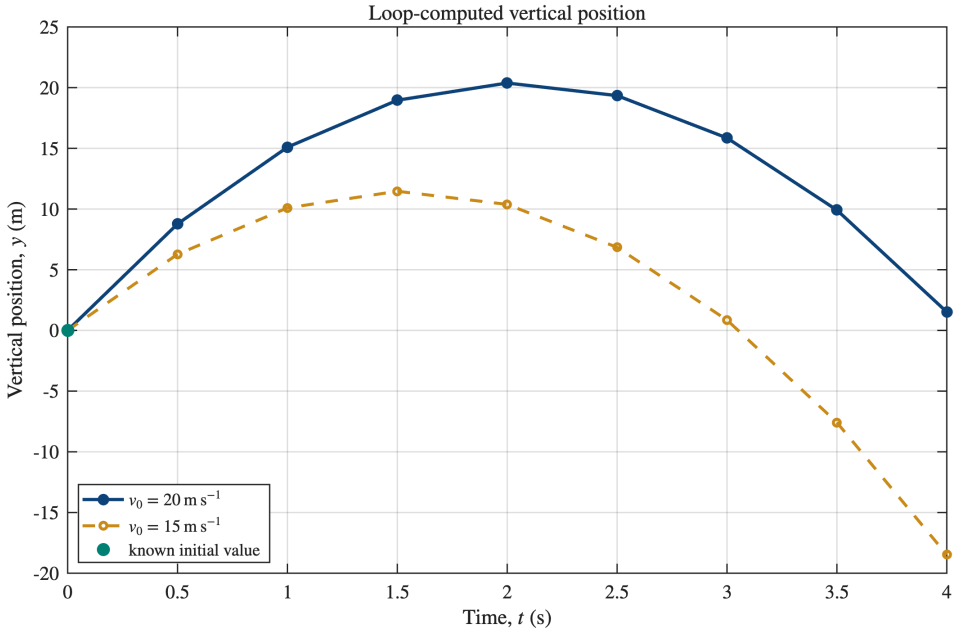



Figure 1: Loop-computed vertical position for the locked lecture model. The lower launch speed produces a lower sampled peak while the initial value remains unchanged.

**Parameter prediction.** Before running the second loop, predict that lowering $v_0$ reduces the maximum height. Only the launch-speed input changes; the loop structure does not.

## 8. One Core validation: the known initial value

At the first stored time, $t = 0$, the model must reproduce its stated initial condition. This value is known before the loop runs.

Listing 5: Executable initial-value validation

```
assert(abs(y_m(1)-y0_m) < 1e-12, ...
    'Initial-position validation failed.')
```

**Validation result.** The first stored value is $y(0) = 0\,\mathrm{m}$, matching $y_0 = 0\,\mathrm{m}$. The assertion passes in a fresh MATLAB session.

The check is powerful but not magical. A wrong gravity sign still gives $y(0) = 0$ because the time-dependent terms vanish at zero. That defect needs physical reasoning as well.

## 9. Debug by classifying the defect

Different defects need different evidence.

| Defect type | What you may observe | Useful first evidence |
|---|---|---|
| Syntax | MATLAB cannot complete the code structure | error location, surrounding punctuation, missing `end` |
| Array/indexing/operator | code may run but reads/stores the wrong values | index-to-array mapping, sizes, selected entries, operator meaning |
| Physical/logical | code runs but implements the wrong physics | sign convention, units, prediction, known or limiting behaviour |

Use the MATLAB error location when MATLAB supplies one, but do not stop there. Compare the line with the pseudocode and physical model.

### Indexing example

This loop repeatedly overwrites only the first storage location:

```
y_index_defect_m = zeros(size(t_s));
for sample_index = 1:n_samples
    current_time_s = t_s(sample_index);
    y_index_defect_m(1) = y0_m + v0_mps*current_time_s ...
        - 0.5*g_mps2*current_time_s^2;
end
```

The repair is to store the current result in `y_index_defect_m(sample_index)` so each time has a matching position slot.

### Physical-sign example

The line below is legal MATLAB but contradicts the upward-positive model:

```
y_wrong_sign_m(sample_index) = y0_m + v0_mps*current_time_s ...
    + 0.5*g_mps2*current_time_s^2;
```

The positive gravity contribution makes the computed trajectory bend upward. The correct model subtracts the gravity term because gravity points downward while upward is positive.

**Debugging caution.** "The code ran" is not validation. A physically wrong expression can produce smooth, plausible-looking numbers without any MATLAB error.

### Working exposure: loop and vectorised forms

The analytical model can also be evaluated with one vectorised expression:

```
y_vectorised_m = y0_m + v0_mps.*t_s - 0.5*g_mps2.*t_s.^2;
```

Here `t_s` is an array, so element-wise operators are required. In the explicit loop, `current_time_s` is a scalar. The two forms should agree numerically for the same model and inputs.

### Optional stretch: package the calculation as a function

A distinction-level extension is to write a reusable function that accepts the model inputs and returns the position array, then design a second independent test. This is not required for the Core Week 2 route.

## Three takeaways

1. Pseudocode makes the order of a computation visible before MATLAB syntax is introduced.
2. A traceable loop connects one index, one current input, one calculation, and one matching output slot.
3. Debugging combines code evidence with validation and physical reasoning; successful execution alone is not proof of correctness.

**Preparation for the practical.** Be ready to transfer the same loop pattern to cooling, radioactive decay, and oscillation. For each context, state the model and units, trace the first passes, validate a known value, and record what any AI assistance changed or did not change.